

Software Engineering Midterm

Name: _____

- Exam is open note, but closed computer - printed notes are fine, but no laptop/phone/smart watch
- Please write legibly!
- Partial answers get partial credit - explain your reasoning to get credit for all the parts of the answer you know
- Write valid Java code for questions asking for code; you **do not** need to include any import statements (assume the IDE will handle that for you)

1. Imagine you are designing an API for a car driving simulator that supports the following workflows: Users must be able to do 5 things: choose a gear (either forward, reverse, neutral, or 1, 2, or 3), press on the gas, press on the brake, turn on the windshield wipers, and check if the windshield wipers are currently on.

Assuming you have an interface called CarAPI where the API will be written, write a prototype of the client of the API. You do **not** need to write any additional interfaces (including CarAPI itself), just the prototype method.

2. Imagine you're building an open source library, and you're hoping it will be used by Large Existing Project. Large Existing Project already uses a library produced by Small Dependency, which includes a class

```
public class NumberList implements Iterable<Integer>, Closeable
```

Your library currently contains the following interface and method:

```
public interface Summable extends Iterable<Integer>, Closeable {}
```

```
public int addValues(Summable toSum) throws IOException {
    int total = 0;
    try (toSum) {
        for (int value : toSum) {
            total += value;
        }
    }
    return total;
}
```

Without adding a dependency on any of the code in Small Dependency (ie, **without including any reference to the NumberList class**), how would you modify your addValues method so that Large Existing Project could call your method with an instance of the NumberList class?

3. Consider the following interfaces and class:

```
public interface CellPhone {  
  
    public Network getCellNetwork();  
  
}  
  
public interface Network {}  
  
public class AndroidPhone implements CellPhone {  
  
    private final Network network;  
  
    public AndroidPhone(Network network) {  
        this.network = network;  
    }  
  
    @Override  
    public Network getCellNetwork() {  
        return network;  
    }  
}
```

Write a **smoke test** for the AndroidPhone class

4. Imagine you're creating a franchise instance of a CoffeeShop, where all prices are increased by \$1 (this franchise might be located in an airport). For reference, the CoffeeShop interface looks like:

```
public interface CoffeeShop {  
  
    public int getPriceRegular();  
  
    public int getPriceLatte();  
  
}
```

(a) What design pattern could you use to solve this problem?

(b) Write the code to apply the design pattern from (a) to solve this problem:

- (c) Suppose you wanted to create a `FranchiseCoffeeShop` class that could increase or decrease prices by amounts other than \$1 (although the amount would still be fixed per instance). How could you modify your code from (b) to accomplish this?

5. Imagine you have the following API for a music streaming service. What would you change about the API (and what code would you replace it with) to make the API more flexible for future changes? Assume all referenced types exist as interfaces.

```
public interface MusicStreaming {  
  
    public void playSong(SongRequest request);  
  
    public Playlist loadMostRecentPlaylist(UserId userIdentifier);  
  
}
```

6. Imagine you're writing a printer management program, and you want to ensure that your print queue is resilient to failures. Write a failure test that verifies that a document that fails to be printed once by throwing an exception and once by returning false can still be successfully printed on the 3rd try, and that all this is gracefully handled by `sendNextItemInQueue()`. You may use any testing technique we've covered in class.

Code for reference:

```
public interface Document {}

public class PrintDriver {
    public boolean sendItemToPrinter(Document toPrint) {
        testingHook();
        return sendItemToPrinterInternal(toPrint);
    }
    private boolean sendItemToPrinterInternal(Document toPrint) {
        /** implementation omitted */
    }
    private void testingHook() {
        // nothing in production
    }
}

public class PrintQueue {
    public static final int NUM_TRIES = 3;
    private final List<Document> queue;
    private final PrintDriver driver;

    public PrintQueue(PrintDriver driver) {
        this.queue = new ArrayList<>();
        this.driver = driver;
    }

    public void addDocumentToQueue(Document toPrint) {
        queue.add(toPrint);
    }
}
```

```

public void sendNextItemInQueue() {
    Document toPrint = queue.get(0);
    boolean success = false;
    for (int i = 0; i < NUM_TRIES; i++) {
        try {
            success = driver.sendItemToPrinter(toPrint);
            if (success) {
                queue.remove(0);
                break;
            }
        } catch (RuntimeException e) {
            // nothing special, just log
            e.printStackTrace();
        }
    }
    if (!success) {
        throw new IllegalStateException("Failed to send item to
printer");
    }
}
}

```


7. Imagine you're reviewing the following code. What's a code review comment you might leave about the design of SearchAPI, specifically the method signatures?

```
/** Represents a user-specified search query */
public interface SearchQuery {
    String getQuery();
}

/**
 * Allows a client to page through an arbitrarily large number of
 * search results
 */
public interface SearchAPI {
    List<String> getPageOfResults(SearchQuery query);
}
```